

COMP3320/6464 Week 4, Lecture 1

Rhys Hawkins

Semester 2, 2024

Agenda

- Memory Access
- Benchmarks and Performance Prediction
- Next week SIMD

Memory Access

An infinitely fast CPU still must load/store data from/to memory

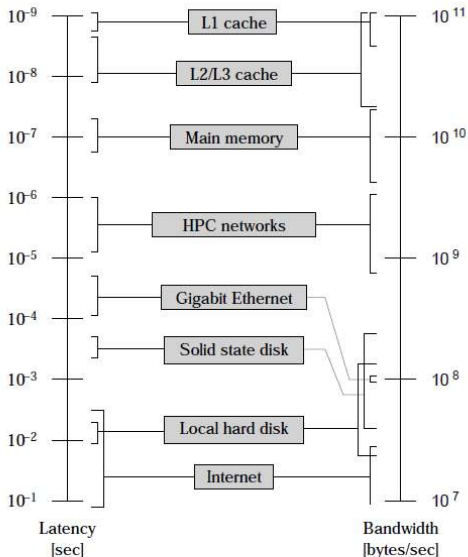
- Great algorithms/code with poor memory accesses will perform poorly
- Many problems are memory bound rather than FLOPs bound

Typical memory access times

Intel Core i7 Xeon 5500 at 2.8Ghz

	Cycles	Time (~ ns)
Registers	1	0.3
L1 Cache Hit	4	1.5
L2 Cache Hit	10	3.5
L3 Cache Hit	40	15.0
Local DRAM	160	60
Remote DRAM	280	100

Memory Hierarchy



Operations of CPU Cache : Read vs Write

- As a general rule, cache misses for write are more expensive than cache misses for read
- In code where there is a trade off between read stride and write stride, prefer lower write stride.
- Examples of this next week

Machine Balance

Arithmetic performance versus Memory Bandwidth

Theoretical performance rule of thumb

$$B_m = \frac{\text{memory bandwidth [GWords/sec]}}{\text{peak performance [GFlops/sec]}} = \frac{b_{max}}{P_{max}}$$

Machine Balance

Arithmetic performance versus Memory Bandwidth

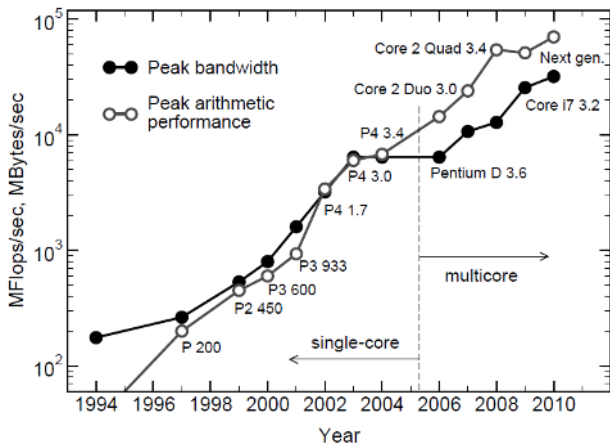


Figure 3.2: Progress of maximum arithmetic performance (open circles) and peak theoretical memory bandwidth (filled circles) for Intel processors since 1994. The fastest processor in terms of clock frequency is shown for each year. (Data collected by Jan Treibig.)

- Widening gap between arithmetic performance and memory bandwidth (access time).
- Memory sizes are also much larger today

Code Balance

Code balance of a loop

$$B_c = \frac{\text{data traffic [Words]}}{\text{floating point ops [Flops]}}$$

Data traffic = all words transferred over slowest data path

Computational Intensity = Inverse of Code Balance

Peak Performance

$l \equiv$ lightspeed of a loop

$$l = \min \left(1, \frac{B_m}{B_c} \right)$$

$P \equiv$ performance (Gflops)

$$P = lP_{max} = \min \left(P_{max}, \frac{b_{max}}{B_c} \right)$$

if $l \sim 1$, performance is not limited by bandwidth but other factors, either inside the CPU or elsewhere.

Low Level Benchmarking

- A program that tests some specific feature of the architecture e.g., peak performance or memory bandwidth
- Useful for approximate performance prediction
- Cannot predict the behaviour of real application code

STREAM benchmarks

type	kernel	DP words	flops	B_c
COPY	$A(:) = B(:)$	2 (3)	0	N/A
SCALE	$A(:) = s * B(:)$	2 (3)	1	2.0 (3.0)
ADD	$A(:) = B(:) + C(:)$	3 (4)	1	3.0 (4.0)
TRIAD	$A(:) = B(:) + s * C(:)$	3 (4)	2	1.5 (2.0)

Table 3.2: The STREAM benchmark kernels with their respective data transfer volumes (third column) and floating-point operations (fourth column) per iteration. Numbers in brackets take write allocates into account.

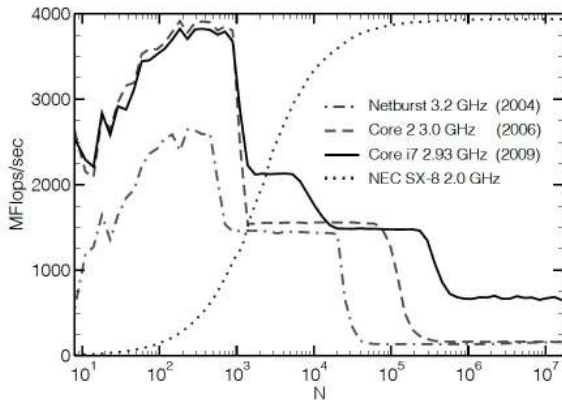
STREAM : Vector Triad Benchmark

```
for (i = 0; i < N; i++) {  
    A[i] = B[i] + C[i]*D[i];  
}
```

- Depending on N , might run very fast and need to be run multiple times.
- For memory speed testing we have to make sure that the arrays are larger than cache sizes

Vector Triad Benchmark

Figure 1.4: Serial vector triad performance versus loop length for several generations of Intel processor architectures (clock speed and year of introduction is indicated), and the NEC SX-8 vector processor. Note the entirely different performance characteristics of the latter.



Example 1: 2D Jacobi Algorithm

Solving diffusion equation

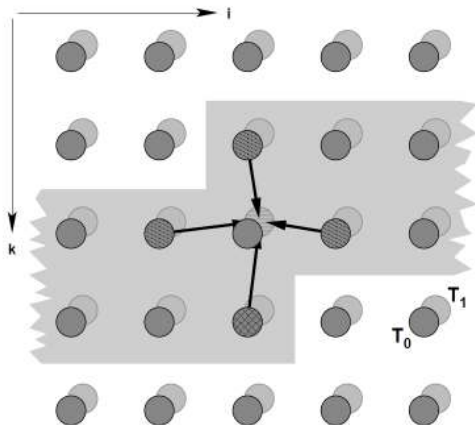
$$\frac{\partial \Phi}{\partial t} = \Delta \Phi ,$$

Discretized using finite difference

$$\frac{\delta \Phi(x_i, y_i)}{\delta t} = \frac{\Phi(x_{i+1}, y_i) + \Phi(x_{i-1}, y_i) - 2\Phi(x_i, y_i)}{(\delta x)^2} + \frac{\Phi(x_i, y_{i-1}) + \Phi(x_i, y_{i+1}) - 2\Phi(x_i, y_i)}{(\delta y)^2}$$

Jacobi Algorithm

Figure 3.5: Stencil update for the plain 2D Jacobi algorithm. If at least two successive rows can be kept in the cache (shaded area), only one T_0 site per update has to be fetched from memory (cross-hatched site).



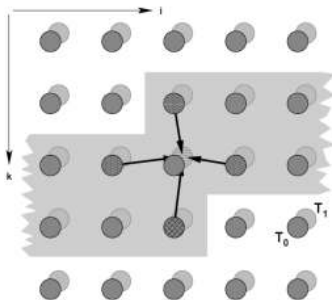
Jacobi Algorithm

```
double phi[2][jmax + 1][imax + 1];
int t0 = 0;
int t1 = 1;
for (j = 1; j < jmax; j++) {
    for (i = 1; i < imax; i++) {
        phi[t1][j][i] = (phi[t0][j][i + 1] + phi[t0][j][i - 1] +
                        phi[t0][j + 1][i] + phi[t0][j - 1][i]) * 0.25;
    }
}
```

- 5 Memory accesses
- 4 FLOPs
- $B_c = 1.25$

Jacobi algorithm: predicted performance

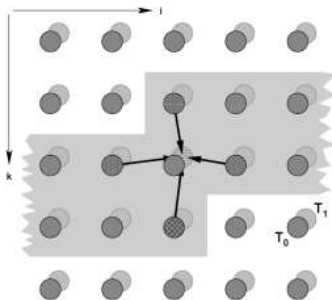
- 4 loads and 1 store per update
- But “downstream neighbour” $\phi(i+1, k)$ is used again from cache for next 2 iterations



- Only 3 loads (of 4) count for the code balance B_c
- $B_c = 1W/F$ (1.25 W/F with write allocate)

Jacobi algorithm: predicted performance

- 4 loads and 1 store per update
- If cache can hold 2 lattice rows, only $\phi(i, k + 1)$ need to be loaded new into cache



- Only 1 load (of 4) count for the code balance B_c
- $B_c = 0.5W/F$ (0.75 W/F with write allocate)

Jacobi Algorithm

Performance measurement

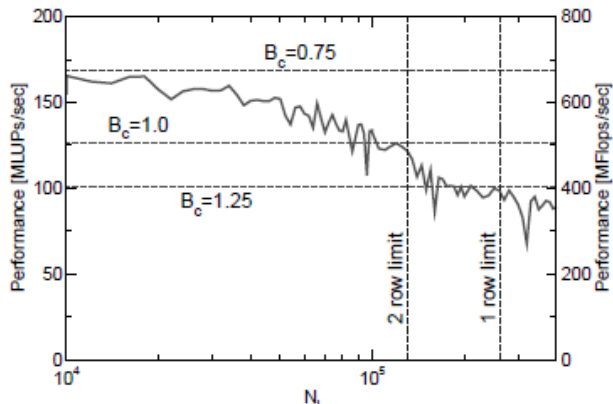


Figure 3.6: Performance versus inner loop length (lattice extension in the i direction) for the Jacobi algorithm on one core of a Xeon 5160 processor (see text for details). Horizontal lines indicate predictions based on STREAM bandwidth.

Summary

- Loop analysis for memory access and FLOPs can give values for code balance B_c
- This can be used to predict performance based on known benchmarks, e.g. STREAM
- Can also given useful feedback in terms of how your code is performing, ie how close is my code performing to the theoretical peak?
- Examples of optimizing memory access for read and write in weeks 5/6.